LEXICON

Lab 1: Guided practice

In this lab you practise what we did in the lecture, on the same API: **https://fakestoreapi.com** (open, no key). You will fetch data, sort and filter it, move your `get_json` function into its own file, and write to the API. Use the `get_json` function from the morning. Remember that it is a fake API, so POST, PATCH and DELETE are not really saved.

Endpoints you need

```
GET   /products
GET   /products/categories
GET   /products/category/<name>
POST  /products
PATCH /products/<id>
```

Tasks

1 **Sort the products by price**
Fetch `/products` with `get_json`. You get a list of dictionaries. Sort the list so the cheapest product comes first and print one line per product with the price and the title. Hint: `sorted(products, key=...)`, where the key function takes one product and returns its price.

2 **Look at the categories**
Fetch `/products/categories` and print the result. Then ask the user to type one of the categories with `input()`. Fetch `/products/category/<name>` and print how many products it has and which one is the cheapest. Think about what happens if the user types a category that does not exist. Note: the category is spelled `jewelery` in this API. That is not your mistake.

3 **Move get_json to its own file**
Create a file called `api_client.py` and move the `get_json` function there. In your lab file, import it with `from api_client import get_json`. Run the lab again. It should work exactly as before. You will reuse this file in Lab 2.

4 **Create and change a product**
Build a dictionary for a new product (title, price, description, image, category, but no id). Send it with `requests.post(..., json=...)` and print the status code and the answer. Then change the price of an existing product, for example product 7, with `requests.patch` and a body that only contains the price. Print the answer. `get_json` only does GET, so here you call requests directly. Remember the timeout and check the status code.

5 **Break it on purpose**
Call your function with a wrong URL, with `timeout=0.001`, and with a made-up domain. For each one, read the message you get. The program must not crash, and every problem should give a short message that a human can read.

Expected for task 4: status 200 and an answer with an id (the fake API answers 200, a real API often answers 201). For PATCH the fake API just sends back what you sent.

Lab 2: Your own tool

In this lab you build a small tool of your own. The user types the name of a city, and the program prints the current temperature and wind speed. You use **Open-Meteo**, which is open and needs no key.

The weather API does not understand city names. It wants coordinates. So the tool needs **two requests after each other**:

Step	What happens	You get
1	Ask the user for a city	A city name, e.g. Stockholm
2	Geocoding API: city name to coordinates	latitude and longitude
3	Forecast API: coordinates to weather	temperature and wind speed, then print

Endpoints you need

```
# 1. City name -> coordinates
https://geocoding-api.open-meteo.com/v1/search
    params: name=Stockholm, count=1

# 2. Coordinates -> weather
https://api.open-meteo.com/v1/forecast
    params: latitude, longitude,
            current=temperature_2m,wind_speed_10m
```

Tasks

1 **Find the coordinates of a city**
Call the geocoding endpoint with `get_json` and the params `name` and `count=1`. **Print the whole answer first** and look at it before you pick out values. You will see a dictionary with a key `results`, which is a list. The first item in that list is your city, with the name, the country, the latitude and the longitude. Then print only the latitude and the longitude.

2 **Get the weather for those coordinates**
Call the forecast endpoint with the params `latitude`, `longitude` and `current` set to `temperature_2m,wind_speed_10m`. Print the whole answer first. You will find a key `current` with the values you want, and a key `current_units` that tells you the units.

3 **Put it together**
Ask the user for a city with `input()`. Use the answer from step 1 for the coordinates, and then call the forecast. Print one line like this (numbers will differ): `Stockholm, Sweden: 8.4°C, wind 12.3 km/h`. Tip: put each of the two requests in its own function, for example `find_city(name)` and `get_weather(lat, lon)`.

4 **Handle a city that does not exist**
What happens if the user types a city that does not exist? Handle it so the program prints a message and does not crash. Hint: print the answer for a city that does not exist, for example `asdfghjkl`, and look at what is different compared with a city that does exist.

Lab 3: Authentication and secrets

In this lab you call an API with a **token**, so the API knows who you are, and you keep that token out of your code and out of Git. You use the GitHub API to print the 5 latest pushed repositories of a user. Without a token GitHub allows 60 requests per hour, with a token 5000.

Endpoint you need

```
GET https://api.github.com/users/<username>/repos
    params: sort=pushed, per_page=5
    header: Authorization: Bearer <token>
```

Tasks

1 **Create a token**
Log in to github.com. Click your profile picture, choose **Settings**, scroll down in the left menu and click **Developer settings**. Choose **Personal access tokens**, then **Tokens (classic)**, then **Generate new token (classic)**. Name it `api-course` and pick a short expiry, for example 7 days. **Do not tick any scopes**, because we only read public information. Click **Generate token** and copy it right away. GitHub shows it only once. (GitHub sometimes renames menus. If something does not match, look for the names in bold.)

2 **Store it safely**
In a new folder, create `.gitignore` **first** and put `.env` in it. Then create `.env` and write `GITHUB_TOKEN=your_token` in it, with no quotes. Do not paste the token in chat, do not take a screenshot of it, and do not share your screen while it is visible.

3 **Call the API with the token**
Load the token with `load_dotenv()` and `os.getenv("GITHUB_TOKEN")`. Send it in the header `Authorization: Bearer <token>`. Ask for a GitHub username with `input()`, call the endpoint above, and print the name of each repository. Use `get_json` from `api_client.py`.

4 **Prove that the token is used**
Look at the response headers. Print `X-RateLimit-Limit` with and without the token. With the token it should say 5000, without it 60. Hint: you need the whole response for this, so call `requests.get` directly.

5 **Break the token on purpose**
Change one character in your token and run the program again. Which status code do you get, and what does the message say? Expected: status 401 and a message about bad credentials. Is that 401 or 403, and what is the difference? Then fix the token again.

6 **Check that the secret is safe**
Run `git init` and then `git status` in your folder. The file `.env` must **not** appear in the list. If it does, your `.gitignore` is wrong. Fix it before you do anything else.

7 **Delete the token**
When you are done, go back to the token page on GitHub and delete the token.

LEXICON

Lab 4: Bonus

Real programs avoid asking an API the same thing again and again, for two reasons: rate limits, and speed. In this lab you add a simple cache to your weather tool. The program saves the answer in a file, and if the file is fresh enough, it reads from the file instead of calling the API.

The idea

```
if the file exists and is younger than 10 minutes:
    data = read the file
else:
    data = call the API
    save data to the file
```

Tasks

1 **Save the answer**
After you get the weather, save the answer to a JSON file with `json.dump`. Run the program and open the file to see what is in it.

2 **Use the file when it is fresh**
Before you call the API, check if the file exists and is **younger than 10 minutes**. If it is, read the data from the file instead. Test it: run the program twice in a row, and make sure the second run does not call the API.

3 **Explain why**
Write two sentences, as a comment in your code, about why this can be useful.

Useful tools

```
import json, os, time

json.dump(data, file)
json.load(file)
os.path.exists(path)
os.path.getmtime(path)   # seconds since 1970
time.time()              # now, in the same unit
```

Stuck? Read the error message from the bottom up, print the data you are working with, and then ask.